CrossTime: A Mobile Application for Smarter Pedestrian Navigation and Traffic Light Awareness

1st Leonardo Ciabattini
Department of Computer Science and Engineering
University of Bologna
Bologna, Italy
leonardo.ciabattini@unibo.it

2nd Alfonso Esposito
Department of Computer Science and Engineering
University of Bologna
Bologna, Italy
alfonso.esposito6@unibo.it

3rd Yasamin Moghbelan
Department of Computer Science and Engineering
University of Bologna
Bologna, Italy
yasamin.moghbelan2@unibo.it

4th Mattia Forlesi
Department of Computer Science and Engineering
University of Bologna
Bologna, Italy
mattia.forlesi2@unibo.it

5th Jennifer Bruno
Department of Computer Science and Engineering
University of Bologna
Bologna, Italy
jennifer.bruno@studio.unibo.it

6th Ivan Zyrianoff
Department of Computer Science and Engineering
University of Bologna
Bologna, Italy
ivandimitry.ribeiro@unibo.it

7th Lorenzo Gigli
Department of Computer Science and Engineering
University of Bologna
Bologna, Italy
lorenzo.gigli@unibo.it

8th Luciano Bononi
Department of Computer Science and Engineering
University of Bologna
Bologna, Italy
luciano.bononi@unibo.it

2025 26th IEEE International Conference on Mobile Data Management (MDM) | 979-8-3315-2569-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/MDM65600.2025.00054

***Abstract*—Mobile Crowd Sensing (MCS) leverages the widespread availability of smart devices to collect and analyze environmental and social data. While MCS has been widely applied in smart cities to optimize vehicle traffic and safety, the needs of pedestrians remain largely unaddressed. To address this need, we propose CrossTime, a sensing application designed to estimate waiting times at pedestrian crossings. Using GPS data, accelerometer readings, and open-source intersection location data, CrossTime autonomously detects when a user is waiting at an intersection. To evaluate its feasibility and accuracy, we conducted a test case on three routes in an urban environment, comparing system-detected waiting times with manually recorded values. Our results show that CrossTime effectively captures pedestrian waiting behavior, although there are some discrepancies due to sensor limitations and environmental factors.**

This research was supported by the PNRR ICSC National Research Centre for High Performance Computing, Big Data and Quantum Computing (CN00000013) under the NRRP MUR program, funded by the NextGenerationEU (Civic Digital Twin). Additional support was provided by the CN-MOST project, Spoke 7.

***Index Terms*—Mobile Crowd Sourcing, Pedestrian mobility, Smart cities, Urban traffic monitoring**

I. Introduction

The rapid development of Internet of Things (IoT) devices has significantly advanced data collection and environmental monitoring [1]–[4]. However, traditional fixed IoT deployments often face challenges such as limited battery life, high infrastructure costs [5], and restricted coverage due to their static positioning [6].

Mobile Crowd Sensing (MCS) offers a compelling alternative, leveraging data from individuals' mobile devices to analyze activities of common interest in urban environments, including traffic management and public safety [5], [7]–[9]. By utilizing the pervasive sensing capabilities of smartphones (GPS, accelerometers, etc.), MCS reduces the need for dedicated infrastructure, offering benefits like lower data collection costs, easier maintenance, and scalability [10], [11].

2375-0324/25/$31.00 ©2025 IEEE
DOI 10.1109/MDM65600.2025.00054

Authorized licensed use limited to: VIT University. Downloaded on August 15,2025 at 07:01:03 UTC from IEEE Xplore. Restrictions apply.

In smart cities, while navigation applications extensively use mobility data for vehicular traffic analysis [6], the specific information needs of pedestrians, particularly regarding traffic signal waiting times, remain largely underserved. Obtaining precise traffic light timing information is often challenging as municipalities seldom share this data openly; furthermore, many pedestrian crossings lack signal control, necessitating a sensing approach. This lack of readily available data hinders smart services that could benefit pedestrian mobility and contribute to sustainable urban living [1], [2], [12]–[14].

CrossTime addresses this gap by providing pedestrians with estimates of mean waiting times at intersections, valuable information for improved route planning. Leveraging smartphone sensors and crowdsourced data, CrossTime makes several key contributions:

- A mobile sensing system that automatically detects when users are waiting at pedestrian crossings using smartphone accelerometer and GPS data.
- A set of algorithms for activity recognition and crossing detection tailored to accurately measure waiting times.
- Integration with OpenStreetMap for identifying nearby intersections and relevant road geometry.
- An evaluation framework demonstrating the system's accuracy in measuring waiting times across various urban scenarios.

CrossTime employs an opportunistic sensing approach, requiring minimal user interaction. The application operates in the background, collecting data efficiently while considering user privacy through appropriate data handling techniques.

The paper is organized as follows: Section II discusses related works on pervasive systems, IoT, and MCS. Section III introduces CrossTime's architecture. Section IV presents the sensing algorithms. Section V proposes an experimental evaluation of feasibility and performance. Finally, Section VI presents conclusions and outlines future work.

II. Related Works

The widespread adoption of IoT has led to Personal IoT (PIoT), integrating devices and sensors for personal environmental interactions. In urban mobility contexts, pervasive computing systems are crucial for smart transportation, adapting to changes based on user needs in real time [15].

Crowdsensing enables increasingly intelligent cities through various applications, such as SmartRoad [16] for identifying traffic signs, GreenDrive [17] for advising drivers on optimal speeds, and CrowdWatch [18] for detecting obstacles.

Despite reliance on fixed-time signals in many cities, AI-driven approaches have emerged for traffic optimization. Research in this area includes combining ML and IoT for dynamic timing using Multi-Agent Reinforcement Learning [19], modeling traffic flow as a Markov Decision Process [20], and integrating LSTM-based prediction with kinematic wave theory [21].

While these studies advance vehicle traffic optimization, they often neglect pedestrian needs and waiting times at crossings—a technological gap in urban navigation. Current applications prioritize vehicular data, overlooking how knowing expected wait times could help pedestrians select optimal routes (e.g., for runners or commuters). CrossTime aims to fill this gap.

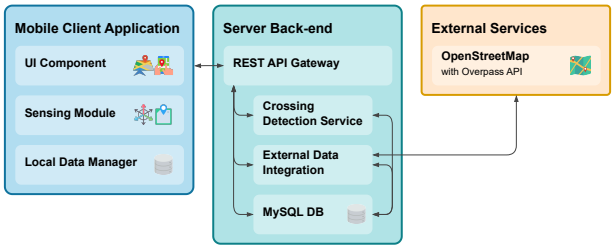


Fig. 1: CrossTime architecture showing the client-mobile application, server-side components, and external services integration.

III. Architecture

This section presents the architecture of CrossTime, a system designed to enhance pedestrian navigation by incorporating waiting times at intersections. CrossTime follows a client-server architecture, leveraging mobile sensing capabilities at the edge and data aggregation services in the cloud. Figure 1 illustrates the overall system architecture, highlighting the main components and their interactions.

The system operates across three main tiers: (1) the mobile edge comprising user smartphones, (2) the server backend providing core functionality, and (3) external services supplying geographical data. This distributed approach balances computational load, minimizes network traffic, and optimizes responsiveness while preserving battery life on user devices. In the following subsections, we provide detailed descriptions of the architecture's key components and their interactions.

A. Mobile Client Application

Residing on users' smartphones, the client application serves as the primary user interface and sensing platform. It provides an interactive map displaying nearby crossings with associated waiting time statistics and suggests routes incorporating these estimates. Key components include:

- **User Interface:** Offers an interactive map, route visualization, user preferences, and history.

Authorized licensed use limited to: VIT University. Downloaded on August 15,2025 at 07:01:03 UTC from IEEE Xplore. Restrictions apply.

- **Sensing Module:** Utilizes GPS for location tracking and the accelerometer for activity classification (WALKING, STILL, UNKNOWN), employing power-efficient strategies like adaptive sampling.
- **Activity Recognition:** Processes sensor readings using the algorithms detailed in Section IV.
- **Local Data Manager:** Caches sensor data and crossing information locally to reduce server load and handle intermittent network connectivity.
- **User Feedback Mechanism:** Optionally prompts users via notifications to validate detected crossing events, contributing to system improvement.

B. Server Backend

The server backend implements the core application logic and data management functionalities. Its main components are:

- **REST API Gateway:** Provides standardized and secure (HTTPS, token-based) endpoints for client-server communication (e.g., retrieving crossing data, submitting sensed data).
- **Crossing Detection Service:** Processes location data submitted by clients to identify nearby crossings using spatial algorithms.
- **Data Aggregation Service:** Computes average waiting times based on data submitted by multiple users for each intersection.
- **External Data Integration:** Interfaces with external services like OpenStreetMap (via Overpass API) to retrieve and maintain up-to-date crossing locations and metadata.

C. Data Storage and Caching

CrossTime utilizes a relational database (e.g., MySQL) for persistent storage of user profiles, crossing locations, individual waiting times, aggregated statistics, user feedback, and application settings. A caching system [22] deployed between the client and server stores frequently accessed data (like nearby crossing information) to improve performance and reduce network traffic, enhancing responsiveness, especially under limited connectivity.

D. Extensibility and Integration

The architecture is designed for future extension and integration. Key extensibility points include standardized API endpoints for third-party integration, a plugin architecture for supporting additional data sources (e.g., public transit feeds), configurable detection algorithms, and open data export capabilities for research and civic applications. This modular approach ensures CrossTime's long-term viability.

IV. Algorithms

This section details the algorithms used in CrossTime to detect crossing events and calculate waiting times at pedestrian intersections, presenting both the underlying mathematical formulations and the procedural logic.

A. Activity Recognition

The CrossTime client app continuously monitors the smartphone's accelerometer to determine the user's current activity state. To enhance reliability and reduce noise, acceleration data is averaged over a time window of t seconds. Let a_i represent the acceleration magnitude at sampling point i, and n be the number of samples in window t. The average acceleration $\bar{a}$ is calculated using Equation 1.

$$\bar{a} = \frac{1}{n} \sum_{i=1}^{n} a_i \tag{1}$$

A user's activity state, initially UNKNOWN, is subsequently classified according to Equation 2, where ϕ represents the activity threshold determined through empirical testing.

$$\text{Activity} = \begin{cases} \text{WALKING}, & \text{if } \bar{a} > \phi \\ \text{STILL}, & \text{if } \bar{a} \leq \phi \end{cases} \tag{2}$$

B. Crossing Event Detection

Algorithm 1 outlines the core logic for handling detected activity states and identifying crossing events based on state transitions.

When a transition from WALKING to STILL occurs (line 2), the application queries the server for the nearest intersection within a predefined radius (line 3). If an intersection is found, a timer is started, the 'waiting' flag is set to true, and the user's side relative to the intersection's road geometry is recorded (lines 5-7).

The user's position relative to the road segment defined by start point $S = (x_s, y_s)$ and end point $E = (x_e, y_e)$ is determined using the sign of the 2D cross product of vectors $\overrightarrow{SE}$ and $\overrightarrow{SU}$ (where $U = (x_u, y_u)$ is the user's position), calculated as shown in Equation 3.

$$\overrightarrow{SE} \times \overrightarrow{SU} = (x_e - x_s) \cdot (y_u - y_s) - (y_e - y_s) \cdot (x_u - x_s) \tag{3}$$

A positive result indicates the user is on the left side, while a negative result indicates the right side (relative to the direction of travel from S to E). This calculation helps detect if a user has crossed the road by comparing their side before and after movement.

When the user resumes walking (detected as a transition to WALKING while the 'waiting' flag is true, line 10), the waiting timer is stopped, and the 'waiting' flag is set to false (lines 11-12). After a brief delay to allow

Authorized licensed use limited to: VIT University. Downloaded on August 15,2025 at 07:01:03 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: CrossTime Activity Handling Logic

```
   Input  : currentActivity ∈ {WALKING, STILL, UNKNOWN}
   Input  : previousActivity ∈ {WALKING, STILL, UNKNOWN}
   Input  : waiting ∈ {true, false}
1  currentPosition ← GetCurrentPosition()
2  if currentActivity = STILL and previousActivity = WALKING
     then
3      intersection ← GetNearestIntersection(currentPosition)
4      if intersection ≠ null then
5          StartWaitingTimer()
6          waiting ← true
7          side ← GetSide(currentPosition, intersection.arch)
               ▷ Record starting side
8      end
9  end
10 else if currentActivity = WALKING and waiting = true then
11     waitingTime ← StopWaitingTimer()
12     waiting ← false
13     Delay(5 seconds)
       ▷ Allow position to stabilize after
         movement
14     newSide ← GetSideOfCurrentPosition(currentPosition,
         arch)
       ▷ Check current side
15     if newSide ≠ side then
           ▷ Check if user crossed the road
16         SaveWaitingTime(user, intersection, waitingTime)
17     end
18     ResetWaitingTimer()
19 end
```

Algorithm 2: Get Nearest Intersection

```
   Input  : position ▷ User's current GPS
            coordinates
   Output: intersection ▷ Nearest valid
            intersection object or null
1  searchRadius ← 10 meters
   ▷ Define search area around user
2  closestIntersection ← null
3  minDistanceFound ← ∞
   ▷ Initialize minimum distance
4  nearbyIntersections ←
     OverpassAPI.GetNearbyIntersections(position, searchRadius)
   ▷ Query OSM
5  foreach intersection ∈ nearbyIntersections do
6      maxDistanceThreshold ← intersection.arch.nLanes · 2 +
         2
7      distance ← HaversineDistance(position,
         intersection.centerPoint)
       ▷ Calculate distance to crossing center
8      if distance < maxDistanceThreshold then
           ▷ Check if within plausible distance
9          if distance < minDistanceFound then
               ▷ Check if this is the closest
                 so far
10             closestIntersection ← intersection
11             minDistanceFound ← distance
12         end
13     end
14 end
15 return closestIntersection
```

the GPS position to stabilize (line 13), the user's current side relative to the road is checked again (line 14). If the side has changed compared to the initially recorded side (line 15), a crossing event is confirmed. The measured 'waitingTime' is then saved to the server associated with the user and intersection (line 16).

C. Intersection Identification

Accurate identification of nearby intersections is crucial for the system's operation. Algorithm 2 details the server-side process for finding the nearest pedestrian crossing relative to the user's reported position using OpenStreetMap data.

This function, executed on the server, queries the OpenStreetMap Overpass API for pedestrian crossings within a 'searchRadius' (e.g., 10 meters) of the user's position (line 4). For each nearby intersection found, it calculates a maximum plausible distance threshold based on the estimated road width (derived from the number of lanes, line 6). It then computes the Haversine distance between the user's position and the center point of the intersection (line 7). If this distance is within the calculated threshold (line 8), it is considered a candidate. The function keeps track of the closest candidate found so far (lines 9-12) and returns it upon checking all nearby intersections (line 13). This ensures the identified intersection is relevant to the user's current location.

Through these algorithms, CrossTime efficiently detects and records waiting times at pedestrian crossings while minimizing false positives and computational overhead. The system's ability to accurately determine when a user has crossed a road enables the collection of valuable data on waiting times, which can then be aggregated to provide improved navigation guidance.

V. Case Study

This section presents the results of the test case conducted to evaluate the performance of CrossTime. Subsection V-A describes the experimental setup, followed by the experimental results in Subsection V-B.

A. Experimental Setup

The goal of the test case was to calculate the accuracy of the system in determining the *waiting time* at intersections. To ensure accurate measurement of this metric, it is essential to determine the exact distance from an intersection and the relative position with respect to the road. To assess performance and accuracy, the test case included two measurements: the time recorded by the application (*sensed time*) and the ground truth time, manually recorded (*real time*). Comparing these values provided insight into the application's accuracy in detecting state transitions.

For our activity recognition algorithm, we set the time window parameter t to 2 seconds, ensuring a balance between responsiveness and stability in state detection.

Authorized licensed use limited to: VIT University. Downloaded on August 15,2025 at 07:01:03 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Error metrics by path

Path ID	Abs. Error				Perc. Error (%)			
	Mean	Min	Max	Std	Mean	Min	Max	Std
1	4.2	1.0	8.0	2.3	34.0	10.0	80.0	21.3
2	1.9	0.0	5.0	2.0	13.7	0.0	31.3	12.3
3	2.8	1.0	8.0	2.2	16.8	6.3	36.4	10.9

The activity threshold ϕ was set to 1.0 m/s², which effectively distinguishes between walking and still states based on our preliminary testing. These values were selected to optimize the system's ability to accurately detect transitions while minimizing false positives.

Experiments were carried out on three different routes in Reggio Emilia, Italy. All potential stops (pedestrian crossings) for each route were documented beforehand to ensure comprehensive data collection during the tests. Figure 2a shows an example map of one route used.

B. Results

This section presents the test case results obtained from the experiments conducted in Reggio Emilia. Observing the data collected on the first route (Figure 2a), we noted that the maximum discrepancy between the sensed time and the real time was 8 seconds, occurring at two distinct locations. This difference is likely attributable to the application's response time in detecting the state change from STILL to WALKING.

The second route (Figure 2b) generally showed more consistent agreement between the sensed and real times compared to the first route. This suggests that the system was able to capture the state transitions more accurately under the conditions encountered on this path.

On the third route (Figure 2c), one intersection exhibited an 8-second discrepancy, similar to the maximum observed in route 1, while other crossings on this route showed smaller errors between sensed and real times.

Table I summarizes the absolute and relative errors calculated for each of the three paths.

Figure 3a displays the distribution of absolute errors. While most errors fall within an acceptable range for navigation purposes, the presence of outliers indicates instances where the detection of state transitions experienced delays. Figure 3b, showing the relative error distribution, confirms this variability, which is likely influenced by environmental factors or momentary sensor inconsistencies.

Analyzing the performance per route, Route 2 (which included 9 crossings) demonstrated the best performance, exhibiting the lowest mean absolute error (1.9s) and standard deviation (2.0s), along with two instances of zero error (perfect accuracy). Route 3 (10 crossings) showed a mean absolute error of 2.8s. Route 1 (11 crossings) displayed the highest variability, with a mean absolute error of 4.2s and outliers reaching up to 8s.

Similarly, for relative errors, route 2 maintained the lowest mean percentage error (13.7%) compared to route 3 (16.8%) and route 1 (34.0%). The maximum percentage error was observed in route 1 (80%), considerably higher than in routes 2 (31.3%) and 3 (36.4%). Overall, across all 30 measured crossings, the system achieves a mean absolute error of 3.0s, which is considered acceptable for practical pedestrian navigation applications where waiting time estimates typically require only moderate precision.

VI. Conclusion and Future Works

This paper evaluated CrossTime, an application leveraging smartphone sensors to incorporate pedestrian waiting times at intersections for improved route planning. We compared app-recorded waiting times with empirical measurements across three scenarios. Results showed reliable performance under certain conditions, with Route 2 exhibiting the lowest errors, highlighting CrossTime's potential for enhanced pedestrian navigation, although accuracy can depend on system response time and environmental factors.

Future work will focus on integrating advanced detection algorithms for more accurate state recognition and potentially distinguishing between waiting types (e.g., traffic light vs. yielding). We also plan to conduct more extensive performance evaluations involving a larger number of subjects and diverse crossing types. Additionally, we will explore the potential for augmenting our data with information from vehicular sources (V2X) and implement the full pedestrian navigation system that utilizes the estimated waiting times to suggest optimal routes.

References

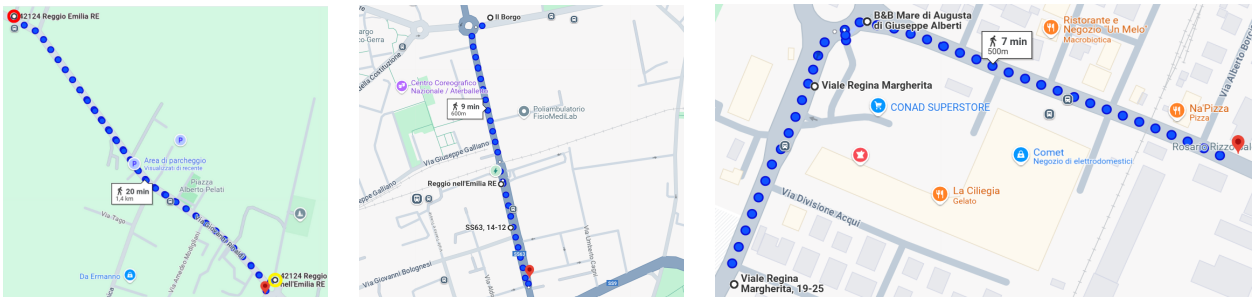
[1] L. Gigli, I. Zyrianoff, F. Zonzini, D. Bogomolov, N. Testoni, M. D. Felice, L. De Marchi, G. Augugliaro, C. Mennuti, and A. Marzani, "Next generation edge-cloud continuum architecture for structural health monitoring," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 4, pp. 5874–5887, 2024.

[2] M. Forlesi, A. Esposito, L. Ciabattini, L. Sciullo, M. Di Felice, S. Kamali, E. Ghini, S. Silvestri, and A. Marzani, "An iot-ai toolchain for structural health monitoring," in *2024 IEEE International Symposium on Measurements Networking (MN)*, 2024, pp. 1–6.

[3] Y. Moghbelan, A. Esposito, I. Zyrianoff, G. Spaletta, S. Borgo, C. Masolo, F. Ballarin, V. Seidita, R. Toni, F. Barbaro, G. Di Conza, F. P. Quartulli, and M. Di Felice, "A smart motor rehabilitation system based on the internet of things and humanoid robotics," *Applied Sciences*, vol. 14, no. 24, p. 11489, Dec. 2024. [Online]. Available: http://dx.doi.org/10.3390/app142411489

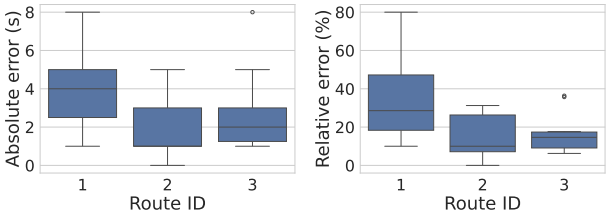
[4] S. Borgo, M. Di Felice, A. Esposito, V. Seidita, G. Spaletta, R. Toni, and I. Zyrianoff, "Poster: Iot and humanoid robotics for next generation motor rehabilitation systems," in *2024 20th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*, 2024, pp. 771–773.

Authorized licensed use limited to: VIT University. Downloaded on August 15,2025 at 07:01:03 UTC from IEEE Xplore. Restrictions apply.



(a) Route 1 (b) Route 2 (c) Route 3

Fig. 2: Walked routes in the city of Reggio Emilia



(a) Absolute Error Boxplot (b) Relative Error Boxplot

Fig. 3: Comparison of Absolute and Relative Errors in waiting times across the three paths.

[5] T. N. Nguyen and S. Zeadally, "Mobile crowd-sensing applications: Data redundancies, challenges, and solutions," *ACM Trans. Internet Technol.*, vol. 22, no. 2, Oct. 2021.

[6] H. Zhu, S. C.-K. Chau, G. Guarddin, and W. Liang, "Integrating iot-sensing and crowdsensing with privacy: Privacy-preserving hybrid sensing for smart cities," *ACM Trans. Internet Things*, vol. 3, no. 4, Sep. 2022.

[7] L. Gigli, F. Montori, G. Galletti, L. Sciullo, and L. Bedogni, "Design and development of a mobile dapp for mobile crowdsensing over evm-enabled blockchains," in *Proceedings of the Fifth ACM International Workshop on Blockchain-Enabled Networked Sensor Systems*, ser. BlockSys '23. New York, NY, USA: Association for Computing Machinery, 2024, p. 21–26.

[8] C.-L. Hu, K.-Y. Lin, and C. K. Chang, "Incentive mechanism for mobile crowdsensing with two-stage stackelberg game," *IEEE Transactions on Services Computing*, vol. 16, no. 3, pp. 1904–1918, 2023.

[9] L. Sciullo, F. Montori, I. Zyrianoff, L. Gigli, D. Tinti, and M. Di Felice, "Designing a hybrid push-pull architecture for mobile crowdsensing using the web of things," in *2023 IEEE International Conference on Smart Computing (SMARTCOMP)*, 2023, pp. 332–337.

[10] Z. Wang, Y. Cao, K. Jiang, H. Zhou, J. Kang, Y. Zhuang, D. Tian, and V. C. M. Leung, "When crowdsensing meets smart cities: A comprehensive survey and new perspectives," *IEEE Communications Surveys Tutorials*, pp. 1–1, 2024.

[11] L. Gigli, F. Montori, M. Zichichi, L. Bedogni, S. Ferretti, and M. Di Felice, "On the decentralization of mobile crowdsensing in distributed ledgers: An architectural vision," in *2024 IEEE 21st Consumer Communications Networking Conference (CCNC)*, 2024, pp. 311–317.

[12] L. Ciabattini, L. Sciullo, A. Esposito, I. Zyrianoff, and M. Di Felice, "Rattle: Train identification through audio fingerprinting," in *2024 IEEE International Conference on Smart Computing (SMARTCOMP)*, 2024, pp. 47–54.

[13] L. Ciabattini, A. Esposito, L. Sciullo, I. Zyrianoff, and M. D. Felice, "Rattle: A framework for train characterization and identification through audio fingerprinting," Online, available at SSRN: http://dx.doi.org/10.2139/ssrn.5189474.

[14] S. Liu, Z. Wan, Y. Yuan, Q. Dong, B. Yang, and F. Hao, "Efficient certificateless blind signature scheme with conditional revocation for mobile crowdsensing within smart city," *IEEE Internet of Things Journal*, vol. 11, no. 9, pp. 15 985–15 997, 2024.

[15] B. Gulzar, S. Sofi, and S. Sholla, "Exploring personalized internet of things (piot), social connectivity, and artificial social intelligence (asi): A survey," *High-Confidence Computing*, 2024.

[16] S. Hu, L. Su, H. Liu, H. Wang, and T. F. Abdelzaher, "Smartroad: Smartphone-based crowd sensing for traffic regulator detection and identification," vol. 11, no. 4, 2015.

[17] Y. Zhao, S. Li, S. Hu, L. Su, S. Yao, H. Shao, H. Wang, and T. Abdelzaher, "Greendrive: A smartphone-based intelligent speed adaptation system with real-time traffic signal prediction," in *2017 ACM/IEEE 8th International Conference on Cyber-Physical Systems (ICCPS)*, 2017, pp. 229–238.

[18] Q. Wang, B. Guo, L. Wang, T. Xin, H. Du, H. Chen, and Z. Yu, "Crowdwatch: Dynamic sidewalk obstacle detection using mobile crowd sensing," *IEEE Internet of Things Journal*, vol. 4, no. 6, pp. 2159–2171, 2017.

[19] S. Damadam, M. Zourbakhsh, R. Javidan, and A. Faroughi, "An intelligent iot based traffic light management system: Deep reinforcement learning," *Smart Cities*, 2022.

[20] A. Merbah and J. Ben-othman, "Optimizing traffic flow with reinforcement learning: A study on traffic light management," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, pp. 7467–7476, 2024.

[21] W. Su, C. Mu, L. Xue, X. Yang, and S. Zhu, "An improved traffic coordination control integrating traffic flow prediction and optimization," *Engineering Applications of Artificial Intelligence*, 2025.

[22] I. Zyrianoff, L. Gigli, F. Montori, L. Sciullo, C. Kamienski, and M. Di Felice, "Cache-it: A distributed architecture for proactive edge caching in heterogeneous iot scenarios," *Ad Hoc Networks*, vol. 156, p. 103413, 2024.

Authorized licensed use limited to: VIT University. Downloaded on August 15,2025 at 07:01:03 UTC from IEEE Xplore. Restrictions apply.